Լաբորատոր աշխատանք 10

Դինամիկ անդամներ և լռելյայն կոնստրուկտոր

Աշխատանքի նպատակը: Ստեղծել դատարկ և տվյալներով օբյեկտներ, անվտանգ փոփոխել դինամիկ զանգվածի չափը:

Տեսական մաս: Լռելյայն կոնստրուկտորը կանչվում է առանց արգումենտների: Դատարկ օբյեկտում n=0 և data=nullptr արժեքները նկարագրում են վավեր վիճակ: Չափը փոխելիս նախ հատկացնել նոր զանգված, փոխանցել տարրերը և միայն դրանից հետո ազատել հինը:

Աշխատանքի կատարման կարգը

1. Կարդալ ընդհանուր պահանջները և նշանակված անհատական տարբերակը:
2. Սահմանել դասի դաշտերը, դրանց սահմանափակումները և մեթոդները:
3. Իրականացնել պահանջվող գործողությունները և ստուգել սխալ մուտքերը:
4. Համեմատել ստուգիչ օրինակի արդյունքը ձեռքով ստացված հաշվարկի հետ:

Ընդհանուր պահանջներ

Դասում պահվում են private double* զանգված, n չափ և p պարամետր: Իրականացված են լռելյայն և պարամետրերով կոնստրուկտորներ, դեստրուկտոր, append(value), removeAt(index), print() և երեք հաշվարկ: Չափափոխումը կատարվում է նոր զանգված հատկացնելով, տվյալները պատճենելով, ապա հինը ազատելով: Պատճենումն ու վերագրումը արգելված են:

Տվյալների չափը՝ 3≤n≤30, չափափոխման դեպքում՝ 0≤n≤30: Թվերը պետք է լինեն վերջավոր: Սխալ չափի, պարամետրի, արժեքի կամ ինդեքսի դեպքում գործողությունը չի կատարվում: Դատարկ վիճակում գումարը և քանակը 0 են, լավագույն ինդեքսը՝ -1:

Կոտորակային ֆունկցիայի ուսումնասիրություն

Տրված է f(x)=1/(x+p): Հաշվել f(x)-երի գումարը, գտնել բացարձակ արժեքով ամենամեծ f(x)-ի ինդեքսը և բացասական f(x)-երի քանակը: Թույլատրելի են վերջավոր x և p արժեքներ, որոնց համար $|x+p|>10^{-9}$: Լռելյայն p=1:

Փորձնական տվյալներ: x={-3,0,2,4}, p=1: Առաջին՝ դատարկ օբյեկտը լրացվում է append-ով, երկրորդը ստեղծվում է տվյալներով: Առաջին օբյեկտից հեռացվում է ինդեքս 1-ի տարրը:

C++ լուծում

```
#include <iostream>
#include <iomanip>
#include <cmath>
#include <stdexcept>
using namespace std;

class ReciprocalStudy {
private:
    double* data;
    int n;
    double p;

    bool validX(double x) const {
        return isfinite(x) && isfinite(p) && fabs(x + p) > 1e-9;
    }

    bool resize(int newN) {
        if (newN < 0 || newN > 30) return false;
        double* temp = (newN == 0) ? nullptr : new double[newN];
        int copyN = (n < newN) ? n : newN;
        for (int i = 0; i < copyN; ++i) temp[i] = data[i];
        delete[] data;
        data = temp;
        n = newN;
        return true;
    }

public:
    // ■■■■■■■■■ ■■■■■■■■■■■■■■ ■■■■■■■ ■■■■■■■
    ReciprocalStudy() : data(nullptr), n(0), p(1.0) {}

    // ■■■■■■■■■■■ ■■■■■■■■■■■■■
    ReciprocalStudy(const double values[], int size, double param)
        : data(nullptr), n(0), p(param) {
        if (size < 3 || size > 30 || !isfinite(p))
            throw invalid_argument("■■■■■ ■■■■ ■■■■ p");
        for (int i = 0; i < size; ++i)
            if (!isfinite(values[i]) || fabs(values[i] + p) <= 1e-9)
                throw invalid_argument("■■■■■■■■■■■■■■■ x ■■■■■■");
        data = new double[size];
        n = size;
        for (int i = 0; i < n; ++i) data[i] = values[i];
    }

    ~ReciprocalStudy() { delete[] data; }

    // ■■■■■■■■■■■ ■■ ■■■■■■■■■■■ ■■■■■■■■■ ■■
    ReciprocalStudy(const ReciprocalStudy&) = delete;
    ReciprocalStudy& operator=(const ReciprocalStudy&) = delete;

    bool append(double value) {
        if (n >= 30 || !validX(value)) return false;
        if (!resize(n + 1)) return false;
        data[n - 1] = value;
        return true;
    }

    bool removeAt(int index) {
        if (index < 0 || index >= n) return false;
        for (int i = index; i < n - 1; ++i)
            data[i] = data[i + 1];
        return resize(n - 1);
    }

    void print() const {
        cout << "x: ";
        for (int i = 0; i < n; ++i) cout << data[i] << " ";
        cout << "\np = " << p << "\n";
    }

    double sum() const {
        double result = 0;
        for (int i = 0; i < n; ++i)
            result += 1.0 / (data[i] + p);
        return result;
    }

    int maxAbsIndex() const {
        if (n == 0) return -1;
        int best = 0;
        for (int i = 1; i < n; ++i)
            if (fabs(1.0 / (data[i] + p)) >
                fabs(1.0 / (data[best] + p)) + 1e-9)
                best = i;
```

```
        return best;
    }

    int negativeCount() const {
        int count = 0;
        for (int i = 0; i < n; ++i)
            if (1.0 / (data[i] + p) < 0) ++count;
        return count;
    }

    int size() const { return n; }
};

void showResults(const ReciprocalStudy& obj) {
    cout << "■■■■■■■ = " << obj.sum() << "\n";
    cout << "■■■■■■■■■ |f(x)|-■ ■■■■■■■ = "
         << obj.maxAbsIndex() << "\n";
    cout << "■■■■■■■■■■ f(x)-■■■ ■■■■■■ = "
         << obj.negativeCount() << "\n";
}

int main() {
    double x[] = {-3, 0, 2, 4};
    double p = 1;

    // ■■■■■■■■■ p=1, ■■■■■■■ ■■■■■■■
    ReciprocalStudy first;
    for (double value : x) {
        if (!first.append(value)) {
            cout << "■■■■■ ■■■■■■\n";
            return 1;
        }
    }

    // ■■■■■■■■ ■■■■■■■■■■ p-■■
    ReciprocalStudy second(x, 4, p);

    // ■■■■■■■ ■■■■■■■■■ ■■■■■■■■■ ■■■■■■■ 1-■
    if (!first.removeAt(1)) {
        cout << "■■■■■ ■■■■■■■\n";
        return 1;
    }

    cout << fixed << setprecision(3);
    cout << "■■■■■■■ ■■■■■■■:\n";
    first.print();
    showResults(first);

    cout << "\n■■■■■■■■ ■■■■■■■:\n";
    second.print();
    showResults(second);
    return 0;
}
```

Ստուգման արդյունք

Առաջին օբյեկտում հեռացվում է x=0 արժեքը, ուստի x={-3,2,4}, p=1։

f(-3)=-0.5, f(2)=0.333..., f(4)=0.2։ Գումարը՝ 0.033..., բացասական արժեքների քանակը՝ 1, իսկ |f(x)|-ի առավելագույն արժեքի ինդեքսը՝ 0։

Երկրորդ օբյեկտում x={-3,0,2,4}, p=1։ Գումարը՝ -0.467..., բացասական արժեքների քանակը՝ 1, իսկ առավելագույն |f(x)|-ի ինդեքսը՝ 0։

Ծրագիրը ստուգում է արժեքների թույլատրելիությունը և չի փոփոխում օբյեկտը հաշվարկների ընթացքում։